

### Módulo 5: Herramientas low-code y no-code con IA (42 h)

**Información de contexto sobre este módulo:**

**Herramientas foco:** Make y n8n (hands-on equilibrado)

**Caso troncal (incremental):** Idealmente, las prácticas de este módulo deben ser llevadas a cabo contra un caso, para que al final del módulo todo el trabajo práctico resulte parte de un proyecto incremental.

En esta guía, cada capítulo incluye tres apartados operativos para facilitar la preparación y la impartición en aula y asegurar que el progreso del grupo es verificable.

- La **Práctica** describe las acciones que el alumnado realiza durante la sesión para aplicar los conceptos del capítulo de forma concreta.
- Los **Entregables** no se plantean como material para corrección, sino como la evidencia mínima de que la práctica se ha completado y de que el resultado es reutilizable en los capítulos siguientes.
- Los **Criterios** de aceptación establecen las condiciones objetivas que debe cumplir esa evidencia para considerarla válida, funcionando como una lista de verificación que permite al docente confirmar que el trabajo es operativo y que el grupo puede avanzar sin arrastrar problemas de diseño o implementación.

### 5.1. Introducción y Contexto Actual (6 h)

**Objetivos de aprendizaje de este apartado:**

Al finalizar este apartado, el alumnado será capaz de:

1. Identificar patrones de automatización low-code/no-code con IA y vincularlos a casos de uso concretos.
2. Definir un formato estándar de datos en JSON para un proceso, especificando campos, tipos, listas y metadatos mínimos.
3. Establecer reglas de validación básicas de entrada y salida para asegurar consistencia operativa.
4. Diseñar un flujo completo por evento o por programación, incorporando rutas principal, revisión y error técnico.
5. Definir criterios de éxito (KPIs) y trazabilidad mínima (logs) para medir y depurar el caso troncal.

#### 5.1.1 Panorama: automatización low-code/no-code con IA y casos de uso (2 h)

**Objetivo**

Alinear expectativas sobre qué automatiza un flujo low-code/no-code, dónde encaja la IA y qué patrones se repiten en la práctica.

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

**Aspectos a cubrir:** Delimitar qué se entiende por automatización low-code/no-code en empresa (integración y orquestación entre aplicaciones, no “IA que lo hace todo”) y presentar los patrones de activación más habituales (evento/webhook, programación/batch y monitorización/polling). Aterrizar casos de uso recurrentes y, sobre todo, el papel específico de la IA dentro del flujo (clasificación, extracción, resumen, etiquetado) y sus límites operativos. Introducir el concepto de flujo “operable” (entrada → normalización → decisión → salida → trazabilidad) y anticipar fallos típicos (datos incompletos, credenciales, rate limits, timeouts, salidas de IA no conformes) para que el alumnado entienda desde el inicio que el diseño debe contemplar validación, rutas alternativas y registro.

### Contenidos

- Panorama de automatización en organizaciones: integración entre aplicaciones, orquestación de tareas, sincronización de datos y creación/actualización de registros.
- Tipos de automatización por patrón de activación:
  - Por evento (webhook / “nuevo registro” / “nuevo mensaje”).
  - Programada (batch y consolidación).
  - Monitorización/polling cuando no hay eventos.
- Casos de uso típicos con IA (qué hace exactamente la IA dentro del proceso):
  - Clasificación: prioridad, categoría, enrutado
  - Extracción: entidades, campos estructurados a partir de texto.
  - Resumen: síntesis operativa para tickets/registros.
  - Etiquetado: palabras clave y temas para reporting.
- Arquitectura mínima de un flujo “operable”: entrada → normalización → decisión (reglas/IA) → salida → trazabilidad.
- Riesgos y fallos recurrentes: datos incompletos, credenciales, límites y latencia, salidas IA no conformes, ausencia de logs.

### Práctica:

- **Inventario de procesos:** cada equipo identifica 6 procesos automatizables (reales, no hipotéticos) y los describe con un formato fijo.
- **Priorización:** elegir 1 proceso como **caso troncal** usando criterios simples (valor, riesgo, datos disponibles, complejidad).
- **Definición de alcance:** delimitar qué entra y qué no entra (fronteras del flujo).

### Entregables

1. **Inventario de 6 procesos** (tabla, 1 página). Campos mínimos:
  - Nombre del proceso
  - Entrada (qué dispara el proceso)
  - Salida (qué queda creado/actualizado/enviado)
  - Sistemas implicados (2-4)
  - Frecuencia (evento / diario / semanal)

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

- Dato crítico (sin el cual no puede operar)
  - Riesgo principal (dato sensible, error reputacional, coste, etc.)
  - Métrica simple (qué mejora si funciona)
2. **Selección del caso troncal** (media página):
- Justificación breve (3–5 líneas)
  - Alcance: “incluye / excluye”
3. **Ficha del caso troncal (1 página):**
- Objetivo operativo
  - Entradas y salidas
  - Sistemas
  - Riesgos y métrica principal

### Criterios de aceptación

- El caso troncal tiene **entrada y salida concretas** (no “mejorar productividad”).
- La salida es verificable: registro creado/actualizado o notificación enviada.
- Hay al menos 1 riesgo identificado con sentido.

## 5.1.2 Datos en automatización: JSON, campos, tipos, listas y validación básica (2 h)

### Objetivo

Definir un formato estándar de datos para el caso troncal y establecer validaciones mínimas (entrada y salida) para evitar fallos y ambigüedades.

**Aspectos a cubrir:** Explicar cómo se representan y viajan los datos en estos entornos, usando JSON como formato de referencia: campos, tipos, anidados y arrays, y por qué es crítico acordar un “formato estándar” que todo el flujo respete. Guiar la definición de campos mínimos del caso troncal (identificadores, fuente/fecha, contenido, categoría/prioridad, metadatos de estado) y aclarar la diferencia entre validación de entrada (obligatorios, formato, longitudes, defaults) y validación de salida (especialmente tras IA: claves requeridas, valores permitidos, estructura esperada). Incluir criterios prácticos para normalización (texto y fechas consistentes, manejo de nulos) y para tratamiento de listas (cuándo se procesan elemento a elemento y qué se registra), de modo que el formador pueda preparar ejemplos de entradas buenas/malas y mostrar cómo una mala estructura rompe rutas y registros.

### Contenidos

- JSON como formato de intercambio: claves, tipos (texto, número, booleano), objetos anidados y arrays.
- Diseño del “formato estándar” del caso troncal (modelo mínimo recomendado):
- Identificadores: id de solicitud, fuente, timestamp.
  - Contenido: texto de solicitud, categoría, prioridad.
  - Contacto (si aplica): nombre, email/teléfono con reglas de formato.
  - Metadatos: estado, ruta tomada, motivo de revisión/error.

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

- Validación de entrada: campos obligatorios, formatos (email/teléfono), longitudes mínimas, valores por defecto.
- Validación de salida (especialmente tras IA): presencia de claves, valores permitidos (p. ej., prioridad ∈ baja/media/alta), estructura y coherencia.
- Normalización habitual: limpieza de texto, fechas en un formato único, tratamiento de nulos/ausentes, mapeos consistentes.
- Implicaciones de listas: cuándo aparecen, cómo se procesan (uno a uno vs lote), y qué se registra cuando hay múltiples items.

### Práctica

- **Definir el modelo de datos** del caso troncal: qué campos van a circular por el flujo.
- **Especificar validaciones**: qué es obligatorio, qué formato se acepta y qué valores son permitidos.
- **Preparar dataset de pruebas**: ejemplos de entradas buenas y malas para probar el flujo.

### Entregables

1. **Esquema JSON del caso troncal** (documento corto + ejemplo):
  - Lista de campos con:
    - Nombre del campo
    - Tipo (string/number/boolean/array/object)
    - Obligatorio (sí/no)
    - Valor por defecto (si aplica)
    - Ejemplo
2. **Reglas de validación (entrada y salida)** (checklist):
  - Entrada (antes de procesar): obligatorios, formatos (email/teléfono), longitudes mínimas
  - Salida (después de IA si aplica): claves requeridas, valores permitidos (ej.: prioridad ∈ baja/media/alta)
3. **Dataset de pruebas (10 payloads)**:
  - 6 válidos + 4 inválidos
  - Cada payload debe venir con etiqueta: “esperado: ruta principal / revisión / error”

### Criterios de aceptación

- El JSON incluye solo campos necesarios para operar (no campos “por si acaso” sin uso).
- Las validaciones son accionables (no “que esté bien escrito”).

- El dataset cubre casos reales (faltan campos, formatos inválidos, texto ambiguo).

### 5.1.3 Diseño del proceso: evento vs programación, criterios de éxito, trazabilidad y mapa del caso troncal (2 h)

#### Objetivo

Diseñar el proceso completo (rutas y métricas) y dejar listo el mapa del flujo que se implementará en Make y n8n.

**Aspectos a cubrir:** Asegurar que el alumnado decide el modo de ejecución con criterio (evento vs programación) en función de latencia, volumen y operación, y que traduce esa decisión a un mapa de proceso implementable. Desarrollar el diseño por rutas (principal, revisión y error técnico) indicando disparadores claros para cada una: validación fallida, ambigüedad, fallo HTTP/credenciales/timeout, etc. Definir criterios de éxito medibles (porcentaje de entradas válidas, revisiones, errores técnicos, tiempo de ciclo y completitud del registro) y qué trazas mínimas deben registrarse para auditar y depurar (id de ejecución, timestamp, ruta tomada, motivo, y referencia del registro creado). Cerrar el capítulo con un diagrama suficientemente detallado para que luego se replique en Make y en n8n sin ambigüedad.

#### Contenidos

- Elección del modo de ejecución:
  - Evento: latencia baja, foco en seguridad del endpoint y control de entradas.
  - Programación: consolidación, control de volumen, manejo de repetición e idempotencia básica.
- Diseño de rutas del proceso:
  - Ruta principal (happy path): entrada válida → decisión → registro/acción.
  - Ruta de revisión: falta dato o caso ambiguo → registro “pendiente” + aviso.
  - Ruta de error técnico: fallo de credenciales/HTTP/timeout → registro de incidencia + alerta.
- Criterios de éxito (operativos, no abstractos):
  - % entradas válidas, % revisiones, % errores técnicos.
  - Tiempo de ciclo (entrada → salida).
  - Calidad del registro (campos completos).
- Trazabilidad mínima (qué se guarda y por qué): id de ejecución, timestamp, payload mínimo, ruta tomada, error/motivo, id del registro creado.
- Mapa del caso troncal: definición de nodos/pasos, puntos de validación, puntos de log, y pruebas mínimas por ruta.

#### Práctica:

- Decidir si el flujo se ejecuta por **evento** o por **programación**, justificándolo.
- Dibujar el flujo completo con **rutas**: principal, revisión, error técnico.
- Definir **métricas** y qué se va a registrar (log) para poder medir y depurar.

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

### Entregables

1. **Mapa del flujo** (diagrama):
  - Trigger (evento/programación)
  - Pasos principales (transformación, decisión, salida)
  - 3 rutas mínimas:
    - Principal
    - Revisión (dato incompleto / caso ambiguo)
    - Error técnico (HTTP/credenciales/timeout)
2. **KPIs operativos (3)**:
  - Definición exacta (cómo se calcula)
  - Fuente (de dónde sale el dato: log/tabla)
  - Objetivo (umbral orientativo)
3. **Checklist de aceptación** (pruebas mínimas):
  - 1 prueba "OK"
  - 1 prueba "Revisión"
  - 1 prueba "Error técnico"
  - Para cada prueba: entrada usada + resultado esperado + evidencia requerida

### Criterios de aceptación

- Las rutas están claramente disparadas por condiciones (no "si pasa algo raro").
- El log contempla lo mínimo: id, timestamp, ruta tomada, motivo y referencia al registro creado.
- Las pruebas se pueden ejecutar en clase con datos controlados.

### 5.2. Estrategia y Comparativa: Make vs n8n (6 h)

#### Objetivos de aprendizaje de este apartado:

Al finalizar este apartado, el alumnado será capaz de:

1. Aplicar un marco de decisión para seleccionar Make o n8n según complejidad del flujo, dato, integraciones, operación y gobierno.
2. Traducir un mismo proceso a la terminología y modelo mental de Make y de n8n, identificando equivalencias funcionales.
3. Comparar Make y n8n con criterios técnicos y operativos (control de flujo, datos/listas, HTTP, observabilidad, entornos, portabilidad).
4. Justificar una elección de herramienta para un caso dado, explicitando trade-offs, riesgos y mitigaciones.

#### 5.2.1 Marco de decisión: cuándo elegir Make o n8n (2 h)

##### Objetivo

Disponer de criterios claros para seleccionar herramienta según contexto, restricciones y operación esperada.

**Aspectos a cubrir:** Presentar una metodología de decisión basada en criterios técnicos y operativos, evitando comparativas superficiales por “catálogo de conectores”. Desglosar ejes como complejidad del flujo (ramas, condiciones, reintentos), complejidad del dato (listas, anidados, validaciones), necesidad de HTTP, observabilidad y depuración, gobierno (roles, separación básica de entornos, control de cambios), y volumen/coste. Explicar cómo ponderar estos criterios según contexto (interno vs externo, sensibilidad del dato, frecuencia de cambios, exigencia de trazabilidad) y convertirlos en una elección defendible. El formador debe insistir en el razonamiento: qué se gana y qué se sacrifica con cada herramienta para el caso troncal.

##### Contenidos

- Criterios de selección con impacto práctico:
  - Complejidad del flujo: ramas, condiciones, rutas paralelas, reintentos.
  - Complejidad del dato: estructuras anidadas, listas, normalización y validación.
  - Integraciones: conector disponible vs necesidad de HTTP; autenticaciones habituales.
  - Operación: observabilidad, depuración, gestión de errores, soporte de equipo.
  - Gobierno: accesos, separación básica de entornos, control de cambios y portabilidad.
  - Volumen y coste: ejecución por evento/batch, límites, coste operativo por ejecución.

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

- Método de decisión: ponderación de criterios por caso (interno/externo; sensible/no sensible; alto/bajo volumen).

### Práctica:

- Completar una **matriz de decisión** con criterios ponderados.
- Aplicarla a:
  - un caso interno tipo,
  - un caso externo tipo,
  - y el caso troncal del equipo.

### Entregables

1. **Matriz de decisión** (tabla con pesos y puntuación):
  - Criterios mínimos:
    - Complejidad del flujo
    - Complejidad del dato/listas
    - Integraciones necesarias
    - Observabilidad/depuración
    - Gobierno/entornos
    - Coste/volumen
  - Peso por criterio (suma 100)
  - Puntuación Make y n8n (1–5) por criterio
2. **Recomendación justificada** (media página):
  - Elección para el caso troncal
  - 3 razones vinculadas a criterios
  - 2 riesgos + mitigación

### Criterios de aceptación

- Hay pesos (no todo vale igual).
- La recomendación se basa en criterios, no en preferencias.

Riesgos y mitigaciones son concretos.

## 5.2.2 Modelo mental y terminología comparada (Make/n8n) (2 h)

### Objetivo

Traducir el mismo flujo a los conceptos nativos de cada herramienta y evitar malentendidos de implementación.

**Aspectos a cubrir:** Alinear el “idioma” de ambas herramientas para evitar confusiones posteriores en la práctica. En Make, fijar conceptos como escenarios, módulos, bundles, mapping, router/filters, error handlers y operaciones; en n8n, workflows, nodes, items, expressions, executions, IF/Switch/Merge, Split In Batches y Error Trigger. Reforzar las equivalencias funcionales (qué hace cada pieza, no solo cómo se llama) y explicar cómo cambia el tratamiento de listas (bundles vs items) y la depuración (historial de ejecución y entradas/salidas por paso). El objetivo es que el formador deje claro cómo se traslada el mismo flujo y las mismas rutas del caso troncal a cada entorno sin alterar la lógica.

### Contenidos



# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

- Make: escenarios, módulos, ejecuciones, bundles, mapping, router, filtros, error handlers, operaciones.
- n8n: workflows, nodes, executions, items, expressions, credentials, IF/Switch/Merge, Error Trigger, Split In Batches.
- Equivalencias operativas “paso a paso”:
  - Trigger: módulo trigger (Make) vs trigger node (n8n).
  - Transformación: mapping/funciones (Make) vs set/expressions (n8n).
  - Enrutado: router+filtros (Make) vs IF/Switch (n8n).
  - Listas: bundles + iterator/aggregator (Make) vs items + Split In Batches (n8n).
  - Errores: error handlers (Make) vs Error Trigger/workflow de errores (n8n).
  - Depuración: historial de ejecución por módulo vs executions por nodo/items.
- Implicaciones de diseño: dónde validar, dónde loguear, y cómo asegurar consistencia del “formato estándar”.

### Práctica:

- Construir un **glosario** de equivalencias Make ↔ n8n.
- Reexpresar el caso troncal en el “idioma” de cada herramienta.

### Entregables

1. **Glosario Make ↔ n8n (mín. 15 términos):**
  - Ejemplos obligatorios:
    - Scenario ↔ Workflow
    - Module ↔ Node
    - Bundle ↔ Item
    - Router/Filters ↔ IF/Switch
    - Error handlers ↔ Error Trigger
    - Iterator/Aggregator ↔ Split In Batches + patrón de consolidación
2. **Mini-diagrama doble** (misma lógica, dos terminologías):
  - Versión Make (escenario/módulos/rutas)
  - Versión n8n (workflow/nodos/rutas)

### Criterios de aceptación

- Las equivalencias son funcionales (qué hace), no solo traducción literal.
- El diagrama conserva las mismas rutas y puntos de validación/log.

## 5.2.3 Comparativa Make vs n8n (tabla + taller de defensa por casos) (2 h)

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

### Objetivo

Comparar de forma estructurada y entrenar la defensa de una decisión técnica-operativa.

**Aspectos a cubrir:** Construir una comparativa estructurada con criterios que impacten en diseño y operación: control de flujo, transformaciones, listas/lotes, integración por HTTP y autenticación, observabilidad y diagnóstico, separación de entornos/gobierno, portabilidad (export/import) y mantenimiento. Guiar al alumnado para que cada criterio se traduzca en implicaciones concretas para su caso troncal (por ejemplo, dónde será más claro el enrutado, qué costará más depurar, cómo se gestionan errores). Preparar el taller de defensa para que practiquen argumentación con trade-offs: elección, riesgos asociados y mitigaciones. El formador debe conducir a conclusiones accionables, no a preferencias.

### Contenidos

- Tabla comparativa por criterios clave:
  - Velocidad de prototipado vs control del flujo.
  - Transformación de datos y tratamiento de listas.
  - Integración HTTP y soporte de autenticación.
  - Observabilidad, depuración y registro.
  - Separación de entornos y gobierno básico.
  - Portabilidad (export/import) y control de cambios.
- Taller de defensa por casos: estructura del argumento (criterio → implicación → decisión → riesgos y mitigación).

### Práctica:

- Completar una tabla comparativa con criterios clave.
- Preparar una defensa corta para 2 casos (interno y externo) usando la tabla.

### Entregables

1. **Tabla comparativa** (1-2 páginas) con criterios:
  - Control de flujo
  - Transformación de datos
  - Listas/lotes
  - HTTP (flexibilidad y autenticación)
  - Observabilidad/depuración
  - Entornos/gobierno
  - Portabilidad (export/import)
2. **Argumentario (10-12 líneas):**
  - "Para este caso elegiría X porque..."
  - "Asumo este riesgo... lo mitigaría así..."

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

3. **Defensa oral (3 min)** + preguntas (si el docente lo aplica)

### Criterios de aceptación

- La tabla no es genérica: cada fila tiene implicación real para el caso.
- El argumentario menciona trade-offs (qué se gana y qué se sacrifica).

### 5.3. Make: construcción de automatizaciones (12 h)

#### Objetivos de aprendizaje de este apartado:

Al finalizar este apartado, el alumnado será capaz de:

1. Construir escenarios en Make con disparadores (Scheduler, Webhooks, Watch) y salidas verificables en sistemas externos.
2. Normalizar datos mediante mapping y funciones, asegurando consistencia de texto, fechas y números según un esquema estándar.
3. Implementar rutas de negocio y reglas de paso con Router y Filters, incluyendo ruta de revisión por validación fallida.
4. Integrar servicios sin conector usando HTTP module, aplicando autenticación y parsing de respuestas.
5. Procesar listas con bundles e Iterator/Aggregator, diseñando para eficiencia y control de operaciones.
6. Implementar gestión de errores con error handlers, logging y alertas, y diagnosticar fallos usando el historial de ejecuciones.

#### 5.3.1 Escenarios, módulos y disparadores (Scheduler, Webhooks, Watch...) (2 h)

##### Objetivo

Construir el esqueleto del escenario del caso troncal: disparo, flujo base y registro en destino.

**Aspectos a cubrir:** Introducir la estructura base de un escenario en Make y cómo fluye la información entre módulos durante una ejecución, asegurando que el alumnado comprende qué implica cada disparador: Scheduler para procesos programados, Webhooks para tiempo real y Watch para monitorización cuando no hay eventos. Explicar cómo configurar conexiones/credenciales y qué errores suelen aparecer (autorización, permisos, parámetros), y dejar montado el “esqueleto” del caso troncal con una salida verificable (registro en hoja/tabla). El formador debe insistir en la disciplina de pruebas: payload de ejemplo, ejecución repetible y evidencia del resultado.

##### Contenidos

- Estructura del escenario: encadenamiento de módulos, paso de datos y ejecuciones.
- Disparadores:
  - Scheduler: periodicidad, casos de uso típicos y control de carga.
  - Webhooks: recepción de payload, estructura esperada y pruebas.
  - Watch...: monitorización cuando no existe webhook, implicaciones de latencia.
- Conexiones: alta de credenciales, permisos mínimos, errores frecuentes (401/403).

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

- Flujo base del caso troncal: recibir → mapear mínimo → escribir registro (tabla/hoja) → confirmación.

### Práctica:

- Crear el **escenario base** del caso troncal con trigger y salida.
- Verificar que el escenario se ejecuta y genera evidencia.

### Entregables

#### 1. Escenario base:

- Trigger: Webhook o Scheduler
- Salida: escritura en tabla/hoja (o sistema simple)

#### 2. Evidencias (2 ejecuciones):

- Entrada válida → registro creado
- Entrada con campo faltante → comportamiento observado (aunque aún no esté la ruta completa)

### Criterios de aceptación

- Hay un registro final verificable.
- Se documenta qué payload se envió (ejemplo guardado).

## 5.3.2 Mapping y funciones: normalización de texto/fechas/números (2 h)

### Objetivo

Implementar la normalización del “formato estándar” del caso troncal usando mapping y funciones.

**Aspectos a cubrir:** Profundizar en el mapping como capa de transformación que garantiza el formato estándar: renombrado de campos, composición, valores por defecto y tratamiento de nulos. Repasar funciones clave para limpiar texto, unificar fechas y convertir tipos numéricos cuando sea necesario, manteniendo consistencia para que las reglas posteriores (Filters/Router) funcionen sin excepciones. El formador debe conectar este capítulo con el esquema JSON definido en 5.1.2, verificando que la salida normalizada es estable y que las variaciones de entrada quedan absorbidas por el mapping.

### Contenidos

- Mapping como capa de transformación: renombrado, composición de campos, valores por defecto.
- Funciones aplicadas a necesidades típicas:
  - Texto: limpieza (espacios/caracteres), concatenación, recorte, normalización de mayúsculas/minúsculas.
  - Fechas: parseo y salida consistente.
  - Números: conversiones, redondeos si aplica.

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

- Manejo de ausencias: campos opcionales sin romper el escenario.
- Validación mínima en Make: checks previos a rutas (preparación para Router/Filters).

### Práctica:

- Implementar la normalización al formato estándar definido en 5.1.2.
- Probar con entradas distintas.

### Entregables

1. **Mapping implementado** (en el escenario):
  - Campos renombrados y valores por defecto
  - Fechas a un formato consistente
2. **Tabla antes/después (5 casos):**
  - Payload de entrada (resumen de campos relevantes)
  - Output normalizado (campos del estándar)
  - Regla aplicada (1 línea por caso)

### Criterios de aceptación

- El output respeta el esquema estándar.
- No se “pierden” campos críticos en el mapping.

## 5.3.3 Router y Filters: rutas de negocio y reglas de paso (2 h)

### Objetivo

Configurar rutas de negocio y reglas de paso para enrutado por prioridad/categoría y para revisión por calidad de datos.

**Aspectos a cubrir:** Explicar cómo se construye el enrutado en Make con Router y Filters, y cómo se diseñan reglas de paso que sean claras, verificables y fáciles de mantener. Aterrizar el enrutado del caso troncal: rutas por prioridad (baja/media/alta) y ruta de revisión cuando falla la validación o falta un dato crítico, especificando qué hace cada ruta (salida diferente, canal de notificación, registro específico). El formador debe enfatizar la importancia de un conjunto de pruebas que dispare cada ruta, y cómo interpretar ejecuciones para comprobar que las condiciones están bien definidas.

### Contenidos

- Router: definición de rutas por caso (baja/media/alta, categorías, estados).
- Filters: condiciones simples y compuestas; orden de evaluación; criterios de exclusión.
- Diseño de rutas del caso troncal:
  - 3 rutas por prioridad con salidas diferenciadas (canal o acción).
  - Ruta de revisión por validación fallida o ambigüedad (registro “pendiente”).
- Estrategia de pruebas: dataset de entradas para disparar cada ruta.

### Práctica:

- Implementar las rutas del caso troncal: 3 prioridades + revisión (mínimo).

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

- Probar rutas con payloads diseñados.

### Entregables

1. **Escenario con rutas:**
  - 3 rutas por prioridad (baja/media/alta)
  - 1 ruta de revisión por validación fallida
2. **Dataset de pruebas (mín. 4 payloads) + evidencia:**
  - 1 payload por ruta
  - Captura/log de ejecución por ruta

### Criterios de aceptación

- Cada payload dispara exactamente la ruta prevista.
- En revisión queda un registro “pendiente” o equivalente.

## 5.3.4 Integraciones cuando no hay conector: HTTP module, autenticación y paginación simple (2 h)

### Objetivo

Integrar un servicio externo mediante HTTP module, contemplando autenticación y un caso de paginación básico si aplica.

**Aspectos a cubrir:** Presentar HTTP module como patrón universal en Make para integrar cualquier servicio: cómo estructurar una petición (método, endpoint, headers, body y parámetros), cómo aplicar autenticación típica (API key/token) y cómo interpretar la respuesta (status codes, parsing de JSON y extracción de campos). Incluir tratamiento básico de errores (401/403, 429, 5xx) y cómo encaminar esos fallos hacia rutas de error con registro. Si el caso lo permite, explicar un patrón de paginación simple (page/offset o cursor) para listar recursos sin perder control. El objetivo es que el formador deje claro cómo pasar de “tengo un endpoint” a “tengo un dato normalizado y registrable”.

### Contenidos

- Construcción de llamada HTTP: método, endpoint, headers, body, parámetros.
- Autenticación habitual: token en header, API key; cuidado con exposición de secretos.
- Interpretación de respuesta: códigos de estado, extracción de campos y normalización al formato estándar.
- Manejo básico de errores HTTP: 400/401/429/500 y qué ruta tomar.
- Paginación simple: concepto y patrón básico (offset/page o cursor) para “listar y procesar”.

### Práctica:

- Integrar un endpoint por HTTP.
- Manejar autenticación.
- Extraer y normalizar respuesta.
- (Si aplica) demostrar un patrón de paginación simple.

### Entregables

1. **HTTP module configurado:**

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

- Método, URL, headers/body
- Autenticación (token/API key)

### 2. Parsing:

- Extracción de 2–3 campos relevantes
- Mapeo a formato estándar

### 3. Prueba de error:

- Simulación de 401 o 429 o 500
- Registro de incidencia con el motivo

### Criterios de aceptación

- Hay evidencia clara de request y de response.
- Ante error, no se “rompe en silencio”: se registra y se controla.

## 5.3.5 Listas y rendimiento: bundles, Iterator/Aggregator y diseño eficiente en operaciones (2 h)

### Objetivo

Controlar el comportamiento con listas y diseñar el escenario para eficiencia (menos pasos y menor multiplicación innecesaria).

**Aspectos a cubrir:** Aclarar cómo Make maneja bundles y por qué las listas multiplican ejecuciones y consumo; identificar dónde se generan bundles y cómo se propagan por el escenario. Explicar cuándo usar Iterator para procesar elementos y cuándo Aggregator para consolidar resultados, y cómo diseñar para eficiencia: validar antes de iterar, evitar llamadas repetidas, y reducir módulos en rutas de alto volumen. El formador debe mostrar el impacto de decisiones de diseño en operaciones y en estabilidad, y guiar al alumnado a comparar una versión “v1” con otra “v2” optimizada manteniendo el mismo resultado funcional.

### Contenidos

- Bundles: cómo se generan y cómo se multiplican a lo largo del escenario.
- Iterator: patrón de procesamiento elemento a elemento (cuándo es inevitable).
- Aggregator: recomposición de resultados (resumen, consolidación, salida única).
- Diseño eficiente orientado a operaciones:
  - Validar antes de iterar.
  - Evitar llamadas repetidas a APIs.
  - Reducir módulos en rutas de alto volumen.
- Identificación de “puntos caros”: dónde se consume más por diseño.

### Práctica:

- Procesar una lista (aunque sea simulada).
- Consolidar resultados si aplica.
- Comparar diseño “directo” vs “optimizado”.

### Entregables

1. **Versión con lista:**



# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

- Iterator para procesar elementos
- (Opcional) Aggregator para resumen final

### 2. Comparativa v1 vs v2 (media página):

- Qué cambiaste (pasos/módulos)  
Por qué mejora (menos operaciones, menos llamadas, validación antes)

### 3. Evidencia de ejecución de ambas versiones

#### Criterios de aceptación

- Se entiende qué multiplica bundles y dónde.
- La versión optimizada reduce pasos o llamadas sin cambiar el resultado.

### 5.3.6 Error handlers y operación: rutas de error, alertas, logging y lectura de ejecuciones (2 h)

#### Objetivo

Dotar al escenario de operación mínima: capturar fallos, registrar contexto y facilitar diagnóstico.

**Aspectos a cubrir:** Establecer un estándar de operabilidad en Make: cómo definir error handlers por módulo, qué estrategia aplicar (reintentar, degradar, registrar y notificar) y qué información mínima debe quedar registrada para diagnóstico. Detallar el diseño del logging (campos mínimos, referencia al paso que falló, payload reducido) y criterios para alertas (qué fallos notifican, umbrales, canal). El formador debe dedicar tiempo a leer ejecuciones reales, localizar el módulo origen, interpretar el error y documentar la acción correctiva, de modo que el alumnado interiorice mantenimiento y depuración como parte del flujo.

#### Contenidos

- Error handlers por módulo: rutas de excepción y decisiones (reintentar, degradar, parar).
- Alertas: cuándo notificar y a quién (fallo técnico, fallo recurrente, umbral).
- Logging: campos mínimos (id ejecución, timestamp, payload mínimo, módulo, error, ruta).
- Lectura de ejecuciones: cómo aislar el módulo que falló, qué mirar en input/output y cómo reproducir.

#### Práctica:

- Implementar manejo de errores (ruta de error).
- Registrar el contexto mínimo.
- Disparar una alerta.
- Practicar diagnóstico en el historial.

#### Entregables

##### 1. Ruta de error (Error handler):

- Captura del fallo

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

- Registro de incidencia (tabla/hoja/log)
- 2. **Alerta:**
  - Notificación (correo/Slack/Teams o similar)
- 3. **Ejecución fallida forzada** + evidencia:
  - Qué fallo se provocó
  - Qué se registró
- 4. **Nota de diagnóstico (5 líneas):**
  - Módulo origen
  - Señal observable (status code / mensaje)
  - Acción correctiva propuesta

### Criterios de aceptación

- El log incluye: id ejecución, timestamp, módulo, error y payload mínimo.
- La alerta contiene contexto suficiente para actuar.

### 5.4. n8n: construcción de automatizaciones (12 h)

#### Objetivos de aprendizaje de este apartado:

Al finalizar este apartado, el alumnado será capaz de:

1. Construir workflows en n8n con triggers (Webhook, Schedule/Cron) y nodos de acción conectados a sistemas externos.
2. Transformar datos por items usando JSON y Expressions, depurando por nodo mediante inspección de inputs/outputs.
3. Integrar servicios externos con HTTP Request y credenciales, realizando parsing de respuestas y manejo básico de errores HTTP.
4. Implementar control de flujo con IF/Switch/Merge y rutas alternativas para revisión o validación fallida.
5. Procesar lotes y listas con Split In Batches aplicando patrones de iteración controlada y registro de resultados.
6. Centralizar errores con Error Trigger y mejorar mantenibilidad mediante executions y modularidad básica con sub-workflows.

#### 5.4.1 Workflows y trigger nodes: Webhook, Schedule/Cron y nodos de acción (2 h)

##### Objetivo

Construir el esqueleto del workflow del caso troncal con trigger y salida.

**Aspectos a cubrir:** Introducir el workflow en n8n como unidad operativa y cómo cambian las ejecuciones al activar/desactivar un flujo, usando executions como herramienta de depuración. Explicar triggers principales (Webhook, Schedule/Cron) y su encaje en el caso troncal, junto con nodos de acción para escribir en destino (tabla/hoja/servicio) y la gestión de credenciales. El formador debe asegurar que el alumnado construye el esqueleto equivalente al de Make, con entrada reproducible, salida verificable y capacidad de inspeccionar datos por nodo.

##### Contenidos

- Workflows: borrador vs activo; executions como fuente de depuración.
- Triggers:
  - Webhook Trigger: payload, endpoint y pruebas.
  - Schedule/Cron: periodicidad y casos típicos.
- Nodos de acción: lectura/escritura en destino (tabla/hoja/servicio).
- Credentials: creación, reutilización y fallos habituales.

##### Práctica:

- Replicar el esqueleto del caso troncal en n8n.
- Ejecutar y registrar.

##### Entregables

1. **Workflow base** (Webhook o Schedule/Cron) + escritura en destino

### 2. **2 ejecuciones** con evidencia (válida y con dato incompleto)

#### **Criterios de aceptación**

- Se puede inspeccionar la ejecución y ver el dato por nodo.
- Existe un registro final verificable.

### **5.4.2 Items + JSON + Expressions: mapeo dinámico y depuración por nodo (2 h)**

#### **Objetivo**

Normalizar datos al formato estándar usando items y expressions, y depurar correctamente por nodo.

**Aspectos a cubrir:** Profundizar en items como unidad de dato en n8n y en cómo el JSON viaja entre nodos, destacando el rol de Expressions para mapear, construir campos dinámicos y aplicar valores por defecto. Usar Set como nodo de normalización para alinear el formato estándar del caso troncal, y enseñar una metodología clara de depuración: inspección de input/output por nodo, detección de campos ausentes y verificación de coherencia de tipos. El formador debe conectar esta normalización con las rutas y validaciones que se construirán después para evitar errores aguas abajo.

#### **Contenidos**

- Items como unidad de dato: cómo fluye el JSON y cómo se inspecciona.
- Expressions: referencias a nodos previos, construcción dinámica de campos, valores por defecto.
- Set: normalización, renombrado, selección de campos, estructura estándar.
- Depuración por nodo: lectura de input/output para detectar dónde se rompe el formato.

#### **Práctica:**

- Normalizar a formato estándar con Set + Expressions.
- Mostrar depuración: input/output de nodos clave.

#### **Entregables**

1. **Normalización implementada** (Set + Expressions)
2. **Evidencia de depuración:**
  - Capturas o export mostrando input/output en 2 nodos (uno antes y uno después del Set)

#### **Criterios de aceptación**

- La normalización produce el mismo esquema estándar que en Make.
- Se documenta claramente dónde se transforma cada campo.

### **5.4.3 Integraciones universales: HTTP Request, credenciales y parsing de respuestas (2 h)**

#### **Objetivo**

Consumir un servicio externo con HTTP Request, usando credenciales y extrayendo datos relevantes de la respuesta.

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

**Aspectos a cubrir:** Presentar HTTP Request como mecanismo universal en n8n, cubriendo configuración completa (método, URL, headers, body, parámetros) y manejo de credenciales sin exponer secretos. Explicar cómo parsear respuestas JSON, seleccionar campos relevantes y normalizarlos al esquema estándar, y cómo tratar errores por códigos de estado con rutas controladas. Incluir casos de laboratorio (401, 429, 5xx) para que el alumnado vea patrones de fallo típicos y el formador pueda mostrar cómo se detectan en executions y cómo se documenta el diagnóstico.

### Contenidos

- HTTP Request: método, URL, headers, body, parámetros.
- Credenciales: configuración práctica (token/API key) y cómo evitar exponer secretos.
- Parsing de respuestas: selección de campos, transformación a formato estándar, manejo de errores por status code.
- Casos de laboratorio: 401 (credenciales), 429 (rate limit) y 500 (error servidor) como puntos de diseño.

### Práctica:

- Llamada HTTP funcional con credenciales.
- Parsing y normalización.
- Tratamiento básico de errores.

### Entregables

1. **HTTP Request configurado** (método, URL, headers/body, credenciales)
2. **Parsing** a formato estándar (2–3 campos mínimos)
3. **Prueba de error** (401/429/500) + registro de incidencia

### Criterios de aceptación

- La ejecución muestra claramente request/response.
- El error se captura con una salida controlada (registro/alerta o ruta definida).

## 5.4.4 Control de flujo: IF / Switch / Merge y rutas alternativas (2 h)

### Objetivo

Implementar rutas de negocio y rutas alternativas con control de flujo nativo de n8n.

**Aspectos a cubrir:** Explicar las piezas de control de flujo en n8n y cuándo usar cada una: IF para decisiones binarias, Switch para clasificación en múltiples rutas y Merge cuando se requiere recombinar resultados. Aterrizar el enrutado del caso troncal (prioridad y revisión) con condiciones explícitas y salidas diferenciadas, destacando cómo se validan entradas antes de accionar y cómo se gestionan rutas alternativas cuando falta un dato o un resultado no es válido. El formador debe insistir en pruebas por ruta y en lectura de ejecuciones para confirmar que la lógica coincide con el diagrama definido en 5.1.3.

### Contenidos

- IF: reglas binarias (válido/no válido; urgente/no urgente).
- Switch: clasificación por múltiples casos (prioridad/categoría).
- Merge: recombinar cuando se requiere una salida unificada o consolidación.

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

- Rutas alternativas: revisión por falta de datos, por validación fallida o por salida IA inválida.

### Práctica:

- Implementar enrutado por prioridad y ruta de revisión.
- Recombinar con Merge si el diseño lo requiere.

### Entregables

1. **Workflow con rutas:**
  - Switch por prioridad
  - IF para validación (si falta campo → revisión)
2. **Evidencias (mín. 4 ejecuciones):**
  - baja/media/alta + revisión

### Criterios de aceptación

- Cada ejecución cae donde debe.
- La revisión deja evidencia (registro pendiente + aviso si aplica).

## 5.4.5 Lotes y listas: Split In Batches y patrones de iteración controlada (2 h)

### Objetivo

Procesar colecciones de items con control (volumen, límites de API y tiempos).

**Aspectos a cubrir:** Tratar el procesamiento de colecciones en n8n con Split In Batches, explicando cuándo aparece (listas desde APIs, consolidaciones programadas, alto volumen) y cómo ajustar el tamaño de lote para controlar límites y tiempos. Detallar el patrón de iteración: procesar item, registrar resultado, capturar fallos por lote y consolidar un resumen final (procesados, fallidos, revisión). El formador debe mostrar cómo inspeccionar la ejecución por lotes y cómo asegurar que el flujo es determinista y trazable cuando hay múltiples items.

### Contenidos

- Cuándo aparece el problema: arrays grandes, listas desde API, consolidaciones programadas.
- Split In Batches: tamaño de lote, avance y control de iteración.
- Patrón de iteración: procesar item → registrar resultado → continuar; y resumen/contadores.
- Diseño ante límites: reducir tamaño de lote, espaciar llamadas, ruta de error por batch.

### Práctica:

- Procesar una lista en lotes.
- Registrar resumen final (contadores, fallos).

### Entregables

1. **Split In Batches configurado** (tamaño de lote explícito)
2. **Registro de resumen final:**
  - procesados
  - fallidos

- enviados a revisión

### 3. **Evidencia de ejecución** mostrando iteración por lotes

#### **Criterios de aceptación**

- Se ve el comportamiento por lotes (no todo de golpe).
- El resumen final se guarda en un destino verificable.

### **5.4.6 Robustez en n8n: Error Trigger, executions y modularidad básica con sub-workflows (2 h)**

#### **Objetivo**

Centralizar el tratamiento de errores y mejorar mantenibilidad mediante modularidad básica.

**Aspectos a cubrir:** Consolidar la operabilidad en n8n con un enfoque centralizado de errores: qué captura Error Trigger, cómo estructurar el workflow de errores (notificación + registro + contexto mínimo) y cómo usar executions para localizar el nodo causante comparando input/output. Introducir modularidad básica con sub-workflows cuando el caso lo justifique (normalización, logging, llamadas HTTP reutilizables) para reducir duplicidad y facilitar mantenimiento. El formador debe dejar un patrón replicable de gestión de incidencias y diagnóstico, no una solución ad hoc.

#### **Contenidos**

- Error Trigger: qué captura, cómo se conecta y qué contexto conviene incluir.
- Diseño del workflow de errores: notificación + registro + campos mínimos de diagnóstico.
- Executions: localizar el fallo por nodo, comparar input/output, reproducir con payload.
- Sub-workflows (modularidad básica): separar normalización, logging o HTTP en bloques reutilizables cuando el caso lo justifique.

#### **Práctica:**

- Configurar Error Trigger para centralizar fallos.
- Registrar contexto mínimo y alertar.
- (Opcional) crear un sub-workflow reutilizable (normalización o logging).

#### **Entregables**

1. **Workflow de errores** (Error Trigger):
  - registra: workflow, nodo (si disponible), mensaje, timestamp, payload mínimo
  - alerta: canal definido
2. **Ejecución fallida forzada** + evidencia
3. **Sub-workflow reutilizable** (si aplica):
  - normalización o logging empaquetado para reutilizar

#### **Criterios de aceptación**

- El error llega al Error Trigger con suficiente contexto.
- El flujo de errores deja rastro persistente (log) y no solo una notificación.

### 5.5. IA y operación mínima en producción (Make y n8n) (6 h)

#### Objetivos de aprendizaje de este apartado:

Al finalizar este apartado, el alumnado será capaz de:

1. Integrar un paso de IA en automatizaciones (conector o HTTP) definiendo un contrato claro de entrada y salida esperada.
2. Validar y postprocesar la salida de IA para convertirla en datos operables conforme al esquema estándar del proceso.
3. Implementar mecanismos de fallback (reintento o ruta de revisión) ante salidas no conformes o casos ambiguos.
4. Aplicar un checklist mínimo de operación en producción (credenciales, logs, alertas y control de cambios).
5. Demostrar un mini-proyecto funcional con una ejecución correcta y una ejecución fallida tratada, aportando evidencias.

#### 5.5.1 IA en automatizaciones: conector cuando exista vs HTTP, entradas/salidas esperadas (2 h)

##### Objetivo

Incorporar IA al caso troncal con un contrato claro de entrada/salida, implementable en Make y n8n.

**Aspectos a cubrir:** Definir con precisión el contrato de IA dentro del caso troncal: qué se envía (campos, formato, normalización previa), qué se espera recibir (campos obligatorios como prioridad/categoría y valores permitidos) y cómo se integra el paso (conector si existe o HTTP si se busca control/portabilidad). Explicar dónde se coloca la IA en el flujo (normalmente antes del enrutado) y por qué la validación posterior es obligatoria para evitar que una salida no conforme active acciones incorrectas. El formador debe asegurar que la implementación sea comparable en Make y n8n, manteniendo el mismo contrato de entrada/salida.

##### Contenidos

- Selección del paso IA (para el caso troncal): clasificación de prioridad y/o categoría; resumen operativo opcional.
- Conector vs HTTP: criterios de elección (simplicidad, control, portabilidad).
- Contrato de entrada: campos mínimos, normalización previa, eliminación de ruido, formato de envío.
- Contrato de salida: estructura esperada (prioridad/categoría/resumen), valores permitidos y formato (texto o JSON).
- Ubicación de la IA en el flujo: antes del enrutado, y con validación posterior obligatoria.



# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

### Práctica:

- Definir contrato de entrada/salida de IA para el caso troncal.
- Implementar el paso IA en Make y en n8n con el mismo contrato.

### Entregables

1. **Contrato IA (1 página):**
  - Entrada: campos que se envían y formato
  - Salida: campos obligatorios y valores permitidos (prioridad cerrada)
2. **Implementación IA en Make** + evidencia de ejecución
3. **Implementación IA en n8n** + evidencia de ejecución

### Criterios de aceptación

- La salida IA produce valores utilizables para enrutado.
- Se registra la salida IA en el destino (no queda “solo en el flujo”).

## 5.5.2 Calidad de salida: validación, postprocesado y fallback (reintento o revisión) (2 h)

### Objetivo

Asegurar que la salida IA se convierte en datos operables y que existe un plan B cuando no cumple.

**Aspectos a cubrir:** Establecer un enfoque sistemático para hacer operable la salida de la IA: validación de campos y valores, control de formato (texto vs JSON), y postprocesado para convertir el resultado al esquema estándar del caso troncal. Describir el fallback como parte del diseño, no como excepción: reintento con instrucciones más restrictivas cuando la salida no cumple, o ruta de revisión con registro “pendiente” cuando el sistema no puede decidir con seguridad. El formador debe trabajar con una batería de inputs difíciles para mostrar cómo se activan rutas principal/fallback y cómo queda trazabilidad del motivo y la decisión tomada.

### Contenidos

- Validación de salida IA: campos obligatorios, valores permitidos, formatos, coherencia con el esquema.
- Postprocesado: normalizar la salida a formato estándar (prioridad, categoría, resumen corto), y enriquecer con metadatos (confianza/ruta tomada si se usa).
- Fallback:
  - Reintento con instrucción más restrictiva (p. ej., forzar valores cerrados).
  - Ruta de revisión (registro “pendiente” + aviso) cuando persiste el problema.
- Batería de pruebas: entradas ambiguas, incompletas, contradictorias y “ruidosas”.

### Práctica:

- Validar salida IA (campos, valores, formatos).
- Postprocesar a formato estándar.
- Implementar fallback: reintento o revisión.

### Entregables

1. **Reglas de validación de salida IA** (checklist)
2. **Postprocesado implementado** (mapeo a esquema estándar)

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

### 3. Fallback operativo:

- reintento (con instrucción más restrictiva) o
- ruta de revisión (registro pendiente + aviso)

### 4. Batería de pruebas (5 casos) con resultado y ruta tomada

#### Criterios de aceptación

- Al menos 1 caso dispara fallback y queda registrado.
- La salida final siempre termina en un estado operativo: registrado o pendiente, pero no “desaparece”.

### 5.5.3 Checklist mínimo de producción + cierre con demo del mini-proyecto (credenciales, logs, alertas) (2 h)

#### Objetivo

Cerrar el módulo con un flujo “operable”: credenciales coherentes, logs, alertas y demostración controlada.

**Aspectos a cubrir:** Cerrar el módulo consolidando requisitos mínimos de operación: gestión de credenciales con mínimo privilegio y nomenclatura, control básico de endpoints (webhooks) y validación de entrada, logging persistente con campos mínimos y alertas con criterios claros. Incorporar control de cambios (versión, export y documentación breve) para que el flujo sea transferible y mantenible. El formador debe preparar una demo final que incluya obligatoriamente una ejecución correcta y una ejecución fallida tratada, mostrando evidencias de log y alerta, y asegurando que el alumnado entrega un paquete reproducible (exports + ficha operativa).

#### Contenidos

- Checklist mínimo de producción (en términos aplicables al aula):
  - Credenciales: mínimo privilegio, naming y separación básica (DEV/PROD conceptual o duplicación controlada).
  - Webhooks: control de exposición, validación de entrada, tratamiento de datos sensibles.
  - Logs: registro mínimo de ejecuciones y de incidencias; dónde se guardan y qué incluyen.
  - Alertas: disparadores (fallo técnico, umbral, ruta de revisión) y canal de notificación.
  - Control de cambios: versión, export, breve documentación (qué hace, entradas/salidas, rutas, puntos de fallo).
- Cierre con demo: una ejecución correcta y una ejecución fallida tratada (con evidencia de logs y alertas).

#### Práctica:

- Preparar el paquete final: exports, checklist y demo.
- Ejecutar 2 demostraciones: una OK y una fallida tratada.

#### Entregables

PLAN DE CAPACITACIÓN JOVEN EN INTELIGENCIA ARTIFICIAL Y EMPLEABILIDAD DIGITAL

# Programa Superior en Inteligencia Artificial Aplicada a la Empresa

## Guía del módulo

### 1. **Checklist mínimo de producción** (completado):

- Credenciales (mínimo privilegio + naming)
- Webhooks (validación de entrada + control de exposición)
- Logs (qué se guarda, dónde, formato)
- Alertas (cuándo salta y por qué canal)
- Control de cambios (versión + export)

### 2. **Demo final:**

- 1 ejecución correcta (evidencia)
- 1 ejecución fallida tratada (evidencia de log + alerta)

### 3. **Paquete de entrega:**

- Export Make + export n8n (o híbrido justificado)
- Ficha operativa (1 página): qué hace, entradas/salidas, rutas, puntos de fallo, dónde mirar logs

### **Criterios de aceptación**

- Hay evidencia de operabilidad: logs y alertas funcionando.
- Los exports permiten reconstruir el trabajo sin “configuración oculta” no documentada.